10. Misc.

Contents

Tree Knapsack **1**

FFT / NTT **1**

Segmented Seive **5**

6

Tree Knapsack

```cpp
vector<int> merge(vector<int> &a, vector<int> &b) {
    vector<int> res;
    res.resize(a.size() + b.size() - 1, INF);
    for (int i = 0; i < a.size(); i++) {
        if (a[i] == INF) continue;
        for (int j = 0; j < b.size(); j++) {
            res[i + j] = min(res[i + j], a[i] + b[j]);
        }
    }
    return res;
}
```

FFT / NTT

```cpp
namespace FFT {
    // [1. CONSTANTS] Modulo and its primitive roots for NTT
    const int mod = 998244353; // 998244353 754974721 167772161
    const int root = 3; // 3 11 3
    const int invRoot = 332748118; // 332748118 617706590 55924054

    inline int mul(int x, int y) { return int(x * 1LL * y % mod); }
    inline int add(int x, int y) { return x + y < mod? x + y: x + y - mod; }
    inline int sub(int x, int y) { return x - y < 0? x - y + mod: x - y; }
    int fp(int b, int e) {
        int res = 1;
        while(e) {
            if(e & 1) res = mul(res, b);
            b = mul(b, b), e >>= 1;
        }
        return res;
    }

    // [3. GENERATOR] O(sqrt(mod) log mod) - Run locally to find
```

```
// root/invRoot for a new prime mod
void primitiveRoot() {
    int phi = mod - 1;
    vector<int> fac;
    for(int i = 2; i * 1LL * i < phi; i++) {
        if(phi % i == 0) {
            fac.push_back(i);
            while(phi % i == 0) phi /= i;
        }
    }
    if(phi > 1) fac.push_back(phi);

    for(int g = 2; g < mod; g++) {
        for(int pr : fac) if(fp(g, (mod - 1) / pr) == 1)
            goto bad;
        cout << "const␣int␣root␣=␣" << g << ";\n";
        cout << "const␣int␣invRoot␣=␣" << fp(g, mod - 2) << ";\n";
        return;
        bad:;
    }
    cerr << "404\n";
}

using cd = complex<double>;
double pi = acos(-1);

// [4. FFT CORE] O(N log N) - Used internally by complex multiplication
void fft(vector<cd> &a, bool invert) {
    int n = (int)a.size();
    for (int i = 1, j = 0; i < n; i++) {
        j ^= ((1 << __lg(n - 1 ^ j)) - 1) ^ (n - 1);
        if(i < j)swap(a[i], a[j]);
    }
    double ang = pi * (invert ? -1 : 1);
    for (int len = 1; len < n; len <<= 1, ang /= 2) {
        cd w1(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len * 2) {
            cd w(1), u, v;
            for(int j = 0; j < len; j++) {
                u = a[i + j], v = a[i + j + len] * w, w *= w1;
                a[i + j] = u + v, a[i + j + len] = u - v;
            }
        }
    }
    if(invert) for(cd &x : a) x /= n;
}

// [5. FFT MULTIPLY] O(N log N) - Multiplies polynomials without
// modulo. Returns exact integers.
// Packs 'a' into real and 'b' into imag to save one FFT pass.
vector<int64_t> mul(vector<int> const &a, vector<int> const &b) {
    int N = 1;
    while (N < a.size() + b.size() - 1) N <<= 1;
    vector<cd> t(N);
    for(int i = 0; i < a.size(); i++) t[i].real(a[i]);
    for(int i = 0; i < b.size(); i++) t[i].imag(b[i]);
```

```
    fft(t, false);
    for(auto &x : t) x *= x;
    fft(t, true);
    vector<int64_t> ans(N);
    for(int i = 0; i < N; i++) ans[i] = (int64_t)round(t[i].imag() / 2.0);
    return ans;
}

// [6. WILDCARD MATCHING] O(N log N) - Returns starting indices of
// pattern 't' in text 's'.
// Supports '?' as wildcard. Returns 0-based indices.
vector<int> string_matching(string &s, string &t) {
    if (t.size() > s.size()) return {};
    int n = s.size(), m = t.size();
    vector<int> s1(n), s2(n), s3(n);
    for(int i = 0; i < n; i++) {
        s1[i] = s[i] == '?' ? 0 : s[i] - 'a' + 1; // assign any non-zero number for non '
        s2[i] = s1[i] * s1[i];
        s3[i] = s1[i] * s2[i];
    }
    vector<int> t1(m), t2(m), t3(m);
    for(int i = 0; i < m; i++) {
        t1[i] = t[m - i - 1] == '?' ? 0 : t[m - i - 1] - 'a' + 1;
        t2[i] = t1[i] * t1[i];
        t3[i] = t1[i] * t2[i];
    }
    auto s1t3 = mul(s1, t3);
    auto s2t2 = mul(s2, t2);
    auto s3t1 = mul(s3, t1);
    vector<int> oc;
    for(int i = m - 1; i < n; i++)
        if(s1t3[i] - s2t2[i] * 2 + s3t1[i] == 0)
            oc.push_back(i - m + 1);
    return oc;
}

// [7. NTT CORE] O(N log N) - Used internally by modular multiplication
void ntt(vector<int> &a, bool invert) {
    int n = (int)a.size();
    for (int i = 1, j = 0; i < n; i++) {
        j ^= ((1 << __lg(n - 1 ^ j)) - 1) ^ (n - 1);
        if(i < j)swap(a[i], a[j]);
    }
    for(int len = 2, l2 = 1, w1, w, u, v; len <= n; len <<= 1, l2 <<= 1) {
        w1 = fp(invert? invRoot: root, (mod - 1) / len);
        for(int i = 0; i < n; i += len) {
            w = 1;
            for(int j = 0; j < l2; j++) {
                u = a[i + j], v = mul(a[i + j + l2], w), w = mul(w, w1);
                a[i + j] = add(u, v), a[i + j + l2] = sub(u, v);
            }
        }
    }
    if (invert) {
        int n_1 = fp(n, mod - 2);
        for(int & x : a) x = mul(x, n_1);
```

```
    }
}

// [8. NTT MULTIPLY] O(N log N) - Multiplies polynomials modulo 'mod'.
vector<int> mulMod(vector<int> a, vector<int> b) {
    int N = 1, sz = a.size() + b.size();
    while (N < a.size() + b.size() - 1) N <<= 1;
    a.resize(N);
    b.resize(N);
    ntt(a, false), ntt(b, false);
    for(int i = 0; i < N; i++)
        a[i] = int(a[i] * 1LL * b[i] % mod);
    ntt(a, true);
    a.resize(sz - 1);
    return a;
}

// [9. FWHT AND/OR/XOR] O(N log N) - Bitwise convolutions.
// N must be power of 2.
void fwht_and(vector<ll>& a, bool invert) {
    int n = a.size();
    for (int len = 1; 2 * len <= n; len <<= 1) {
        for (int i = 0; i < n; i += 2 * len) {
            for (int j = 0; j < len; ++j) {
                a[i + j] = (a[i + j] +
                    (invert? -1: 1) * a[i + j + len] + mod) % mod;
            }
        }
    }
}
void fwht_or(vector<ll>& a, bool invert) {
    int n = a.size();
    for (int len = 1; 2 * len <= n; len <<= 1) {
        for (int i = 0; i < n; i += 2 * len) {
            for (int j = 0; j < len; ++j) {
                a[i + j + len] = (a[i + j + len] +
                    (invert? -1: 1) * a[i + j] + mod) % mod;
            }
        }
    }
}
void fwht_xor(vector<ll>& a, bool invert) {
    int n = a.size();
    for (int len = 1; 2 * len <= n; len <<= 1) {
        for (int i = 0; i < n; i += 2 * len) {
            for (int j = 0; j < len; ++j) {
                ll u = a[i + j], v = a[i + j + len];
                a[i + j] = (u + v) % mod;
                a[i + j + len] = (u - v + mod) % mod;
            }
        }
    }
    if (invert) {
        ll inv2 = (mod + 1) / 2;
        ll inv_n = 1;
        for(int i = 1; i < n; i <<= 1)
```

```
                inv_n = inv_n * inv2 % mod;
            for (ll &x : a) x = x * inv_n % mod;
        }
    }

    // [10. CONVOLUTION RUNNER] O(N log N) - Pass fwht_and,
    // fwht_or, or fwht_xor as 'fun'.
    // Solves for C[k] = sum(A[i] * B[j]) where i (op) j = k.
    template<typename F>
    vector<ll> convolution(vector<ll> a, vector<ll> b, F const &fun) {
        int n = 1;
        while (n < max(a.size(), b.size())) n <<= 1;
        a.resize(n), b.resize(n); // Resizes automatically to next power of 2
        fun(a, false);
        fun(b, false);
        for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
        fun(a, true);
        return a;
    }
}
```

Segmented Seive

```
vector<int> simple_sieve(int n) {
    vector<bool> is_prime(n + 1, true);
    vector<int> primes;
    if (n >= 0) is_prime[0] = false;
    if (n >= 1) is_prime[1] = false;
    for (int i = 2; i <= n; i++) {
        if (!is_prime[i]) continue;
        primes.push_back(i);
        if (1LL * i * i <= n) {
            for (ll j = 1LL * i * i; j <= n; j += i) {
                is_prime[j] = false;
            }
        }
    }
    return primes;
}

vector<bool> segmented_sieve_mask(ll l, ll r) {
    if (l > r) return {};
    int root = sqrtl(r);
    while (1LL * (root + 1) * (root + 1) <= r) root++;
    while (1LL * root * root > r) root--;
    vector<int> primes = simple_sieve(root);
    vector<bool> is_prime(r - l + 1, true);
    for (ll p: primes) {
        ll start = max(p * p, ((l + p - 1) / p) * p);
        for (ll j = start; j <= r; j += p) {
            is_prime[j - l] = false;
        }
    }
    for (ll x = l; x <= min(r, 1LL); x++) {
        is_prime[x - l] = false;
```

```
    }
    return is_prime;
}

// returns all primes in [l, r]
vector<ll> segmented_sieve(ll l, ll r) {
    vector<bool> is_prime = segmented_sieve_mask(l, r);
    vector<ll> primes;
    for (ll i = 0; i < (ll) is_prime.size(); i++) {
        if (is_prime[i]) primes.push_back(l + i);
    }
    return primes;
}
```